SenEvent

Accueil Nouvel evenement Se connecter

DAWM · BLOC BACKEND 1/2 · M1 RETEL — DMI/FST/UCAD

Lab 5 — Supabase : projet, schéma SQL & auth

Rapport de réalisation : une base PostgreSQL managée, un schéma à trois tables sécurisé par Row Level Security, le SDK `@supabase/supabase-js` câblé dans React, et un cycle complet d'authentification signup / login / logout.

tag : v0.5-supabase | PostgreSQL | Row Level Security | @supabase/supabase-js | Build ✓ 0 erreur

01 Le schéma SQL

Trois tables applicatives sous le schéma `public`, greffées sur la table `auth.users` fournie par Supabase. Clés primaires, clés étrangères, contraintes `check` et valeurs par défaut sont déclarées en base — la structure ne peut pas être contournée depuis le client.

TABLE	CLÉ PRIMAIRE	COLONNES CLÉS	RLS
`profiles`	`uuid (= auth.users)`	nom, telephone, role	lecture publique, update self
`evenements`	`bigint identity`	titre, categorie, lieu, prix, organisateur_id	lecture publique, insert/update/delete owner
`inscriptions`	`bigint identity`	evenement_id, utilisateur_id	lecture publique, insert/delete self

SUPABASE/MIGRATIONS/001_SCHEMA.SQL

```
create table public.profiles (
  id uuid primary key references auth.users (id) on delete cascade,
  nom text not null,
  telephone text,
  role text not null default 'PUBLIC'
    check (role in ('PUBLIC', 'ORGANISATEUR', 'ADMIN')),
  created_at timestamptz not null default now()
);

create table public.evenements (
  id bigint generated always as identity primary key,
  titre text not null check (char_length(titre) >= 3),
  categorie text not null check (categorie in
    ('concert','expo','conference','atelier','soutenance')),
  lieu_nom text not null,
  date_debut timestamptz not null,
  prix integer not null default 0 check (prix >= 0),
  image_url text,
  organisateur_id uuid references public.profiles (id) on delete set null,
  created_at timestamptz not null default now()
);

create table public.inscriptions (
  id bigint generated always as identity primary key,
  evenement_id bigint not null references public.evenements (id) on delete cascade,
  utilisateur_id uuid not null references auth.users (id) on delete cascade,
  created_at timestamptz not null default now(),
  unique (evenement_id, utilisateur_id)
);
```

02 Row Level Security

Dès qu'on active RLS, l'accès est refusé par défaut (*deny-by-default*) tant qu'aucune politique n'autorise l'action. On écrit alors une politique par cas d'usage. Distinction clé : `using(...)` filtre les lignes existantes visibles/modifiables ; `with check(...)` contraint les valeurs d'une ligne insérée ou modifiée.

SUPABASE/MIGRATIONS/002_RLS.SQL — EXEMPLE TABLE EVENEMENTS

```
alter table public.evenements enable row level security;

-- Lecture publique (meme anonyme)
create policy "evenements_select_public"
  on public.evenements for select using (true);

-- Insertion pour un utilisateur authentifie, qui devient organisateur
create policy "evenements_insert_auth"
  on public.evenements for insert to authenticated
  with check (organisateur_id = auth.uid());

-- Modification / suppression par le proprietaire seulement
create policy "evenements_update_owner"
  on public.evenements for update to authenticated
  using (organisateur_id = auth.uid());
```

Les tables `profiles` et `inscriptions` suivent le même modèle : lecture publique, écriture restreinte au propriétaire via `id = auth.uid()` ou `utilisateur_id = auth.uid()`. La contrainte `unique(evenement_id, utilisateur_id)` interdit la double inscription au niveau de la base.

03 Le client Supabase dans React

Le SDK est instancié une seule fois depuis les variables d'environnement de Vite (préfixe `VITE_`), exposées via `import.meta.env`. Le fichier `.env.local` reste hors de Git (`.gitignore`) : la clé n'apparaît jamais dans l'historique.

SRC/LIB/SUPABASE.JS

```
import { createClient } from "@supabase/supabase-js";

const url = import.meta.env.VITE_SUPABASE_URL;
const anonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

if (!url || !anonKey) {
  throw new Error("Variables Supabase manquantes. Verifiez .env.local");
}

export const supabase = createClient(url, anonKey);
```

APP.JSX — ÉCOUTE DE SESSION

```
useEffect(() => {
  // Session actuelle au montage (lue depuis le localStorage)
  supabase.auth.getSession().then(({ data }) => setSession(data.session));

  // Ecouter login / logout / refresh de token
  const { data: subscription } = supabase.auth.onAuthStateChange(
    (_event, newSession) => setSession(newSession)
  );

  return () => subscription.subscription.unsubscribe();
}, []);
```

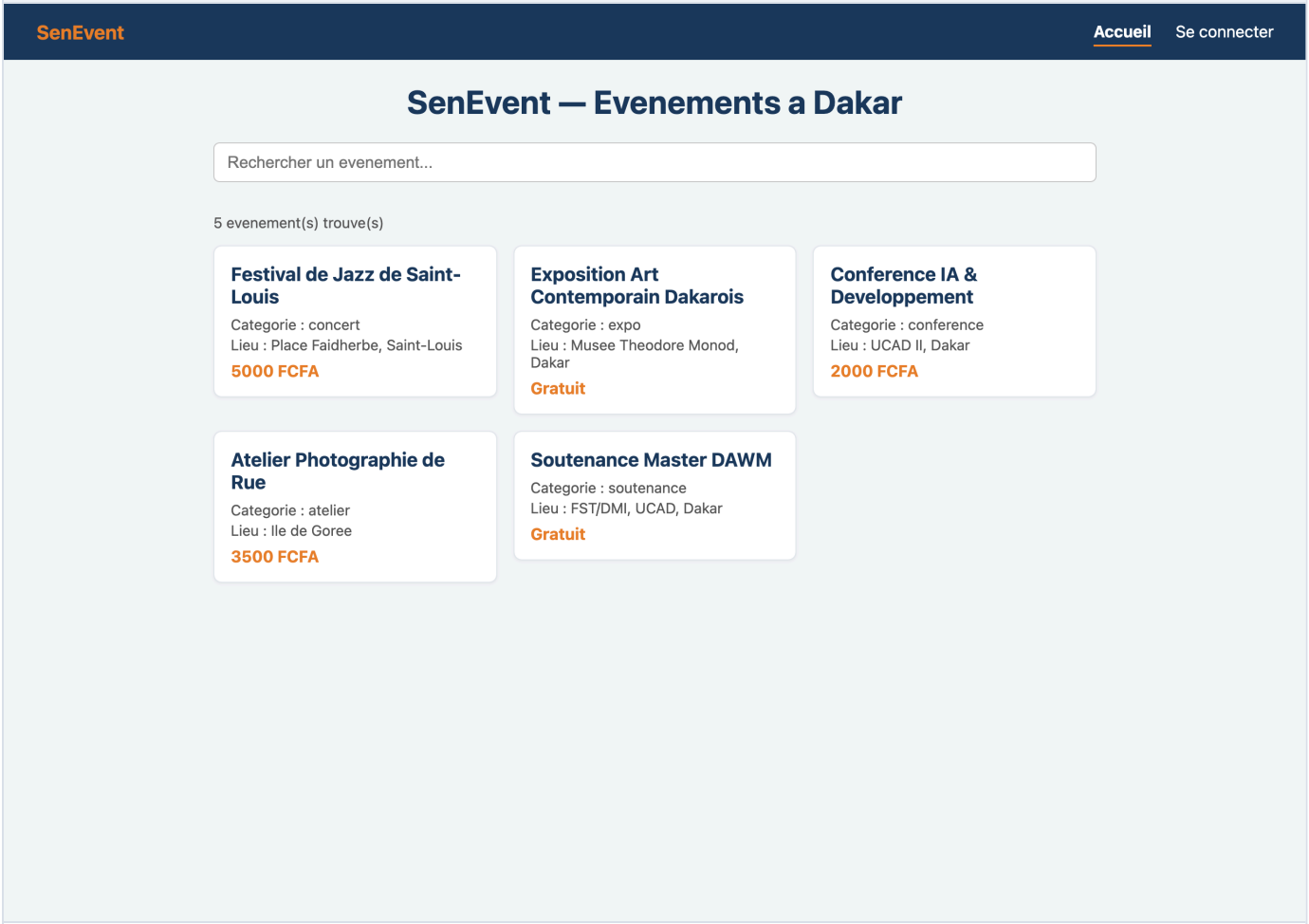
`getSession()` lit la session en cache au montage ; `onAuthStateChange` maintient l'état `session` synchronisé à chaque login, logout ou rafraîchissement de token. La fonction de nettoyage désabonne l'écouteur au démontage.

04 L'authentification, en situation

Captures prises sur l'application en fonctionnement, en jouant le parcours complet : état non connecté, connexion, inscription, session active avec NavBar contextuelle, et gestion d'une erreur d'identifiants.

FIGURE 1 /

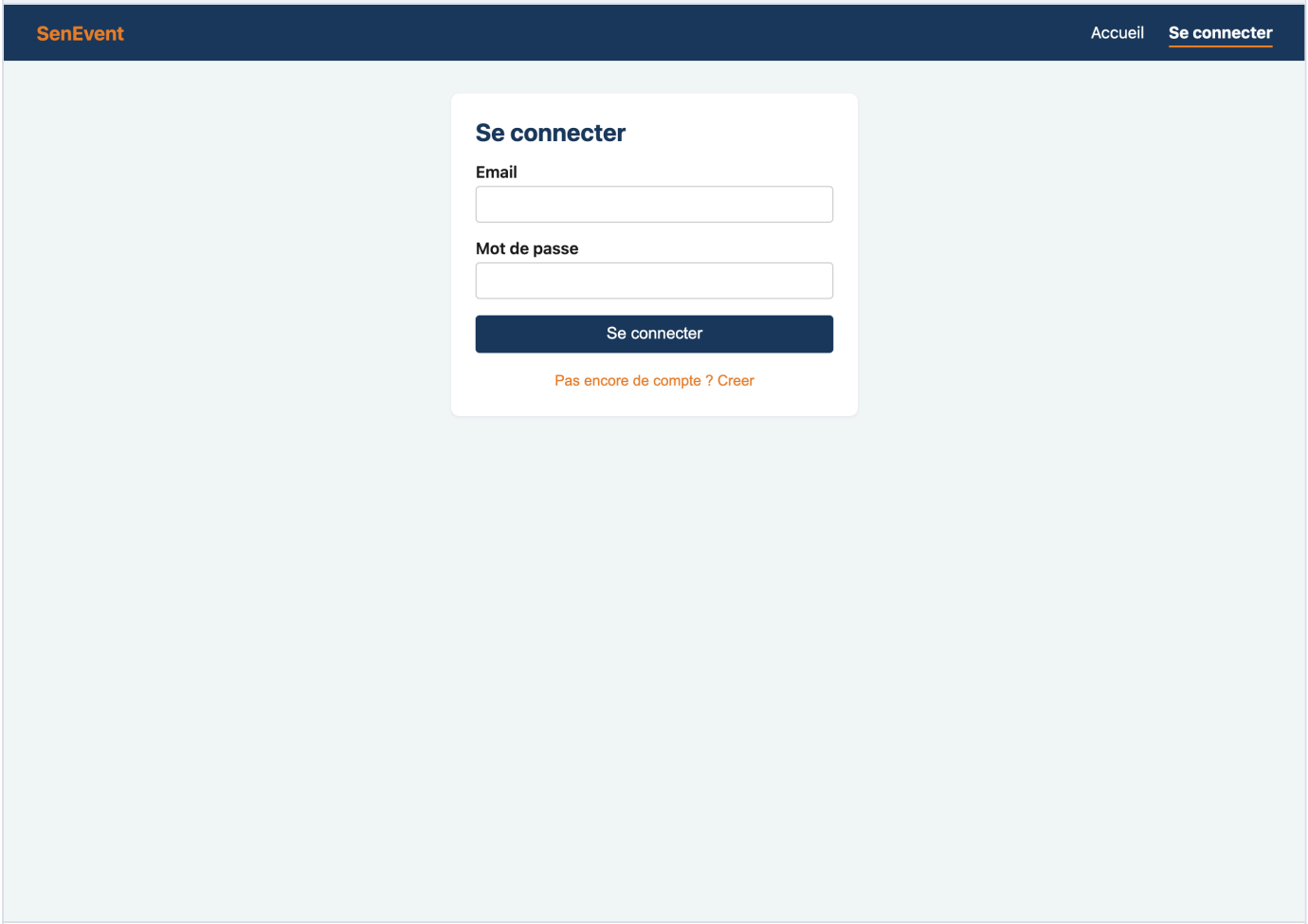
Etat non connecte



Au chargement, `getSession()` ne trouve aucune session : la NavBar n'affiche que « Accueil » et « Se connecter ». Le lien « Nouvel evenement », reserve aux connectes, reste masque.

FIGURE 2 `/auth`

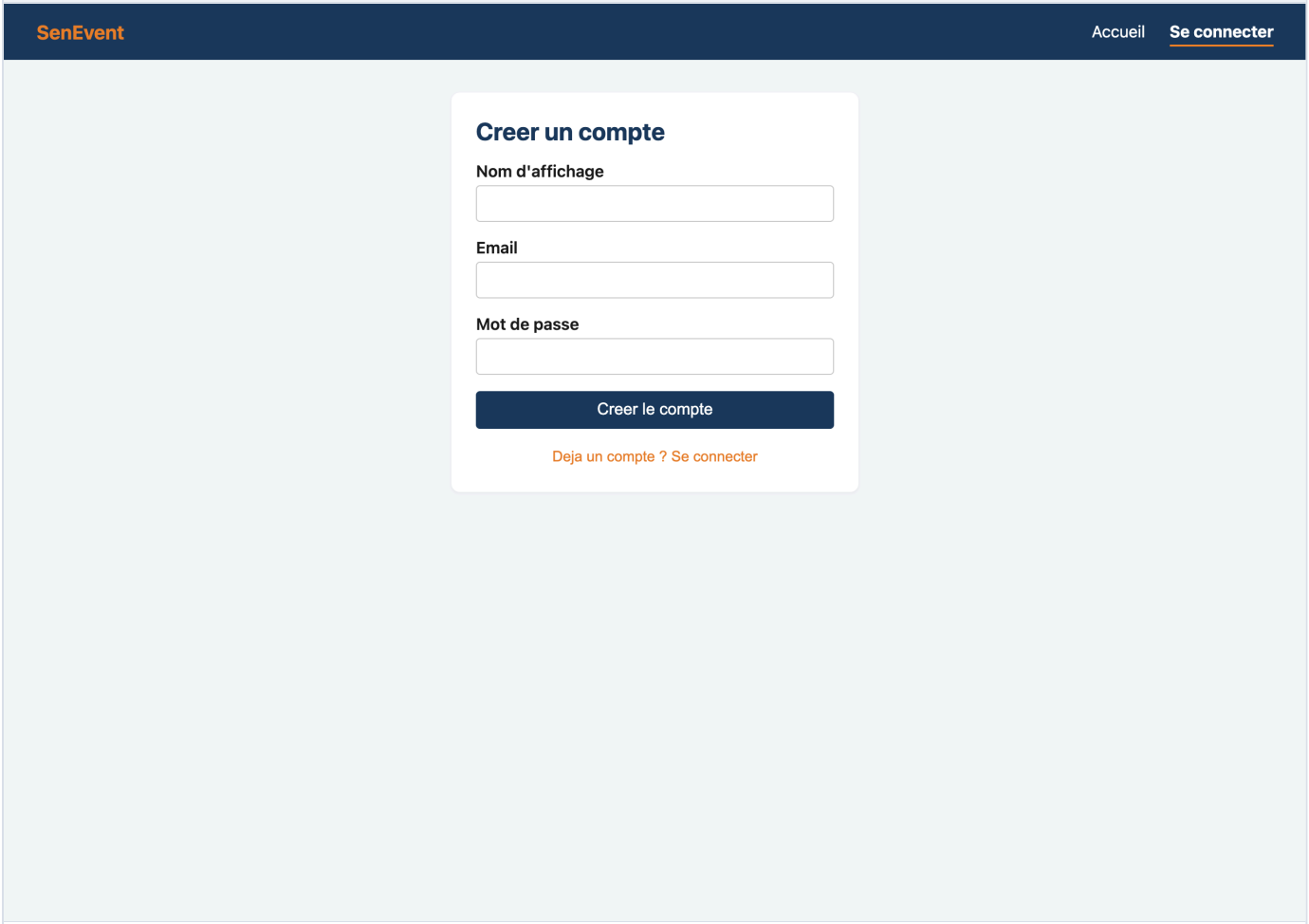
Ecran de connexion



La route `/auth` rend le composant `Auth` en mode `login` : email + mot de passe. La soumission appelle `supabase.auth.signInWithPassword`.

FIGURE 3 `/auth`

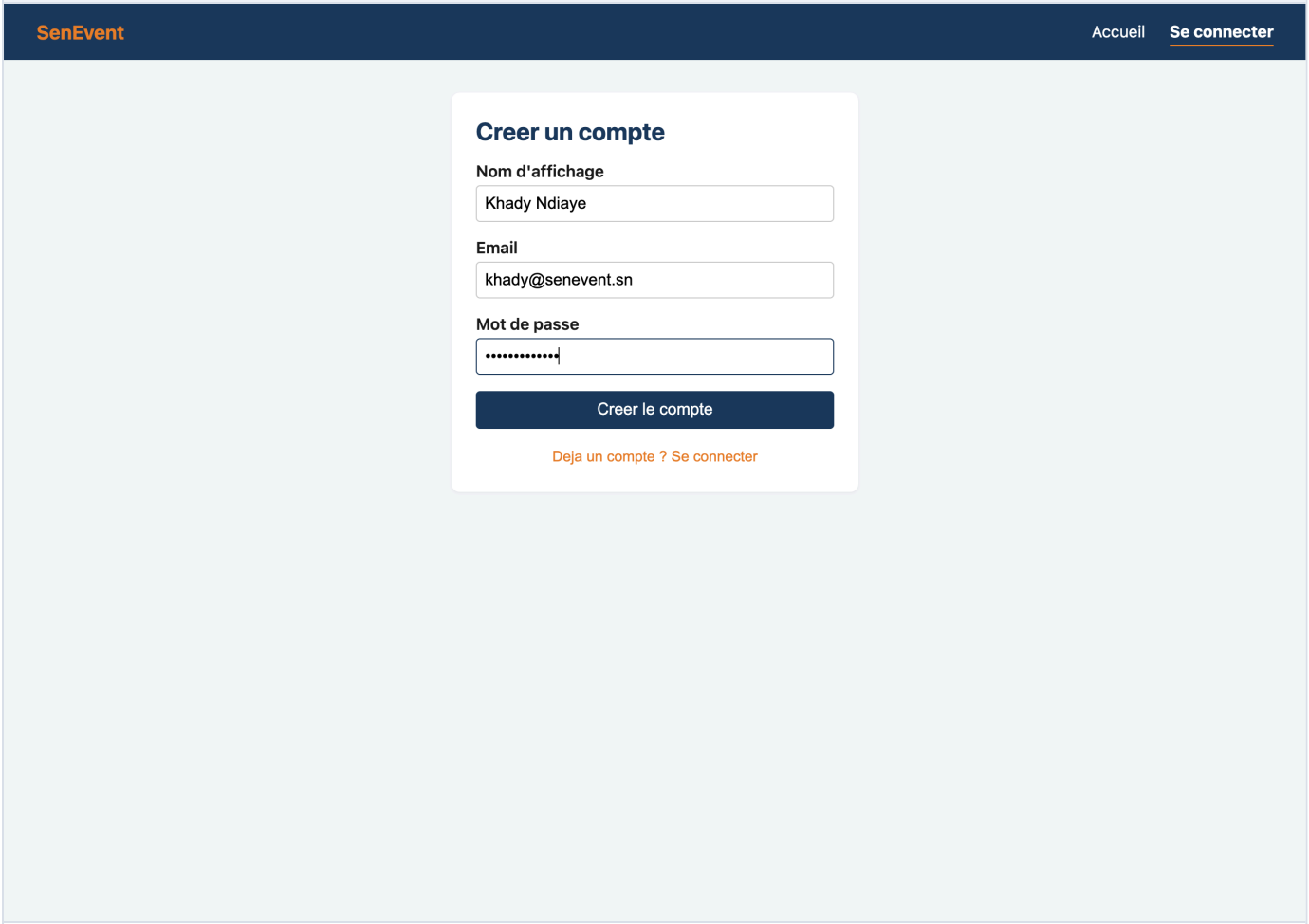
Bascule vers l'inscription



Un clic sur « Pas encore de compte ? Creer » bascule l'etat `mode` vers `signup` : le champ « Nom d'affichage » apparait. Un seul composant gere les deux parcours.

FIGURE 4 `/auth`

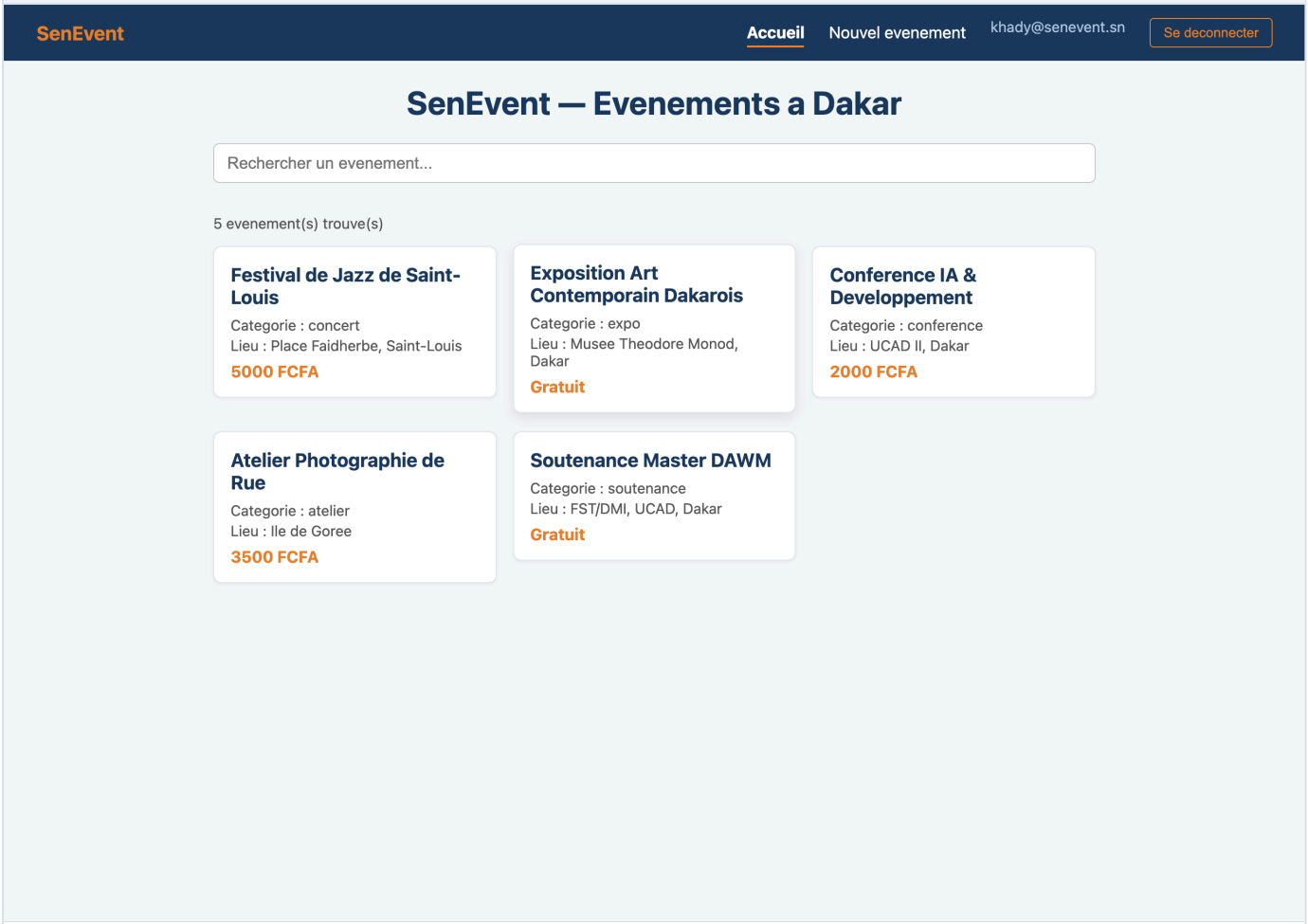
Formulaire d'inscription rempli



Nom, email et mot de passe (6 caracteres minimum, controle par `minLength`) saisis. La soumission enchaine `signUp` puis l'insertion de la ligne `profiles`.

FIGURE 5 /

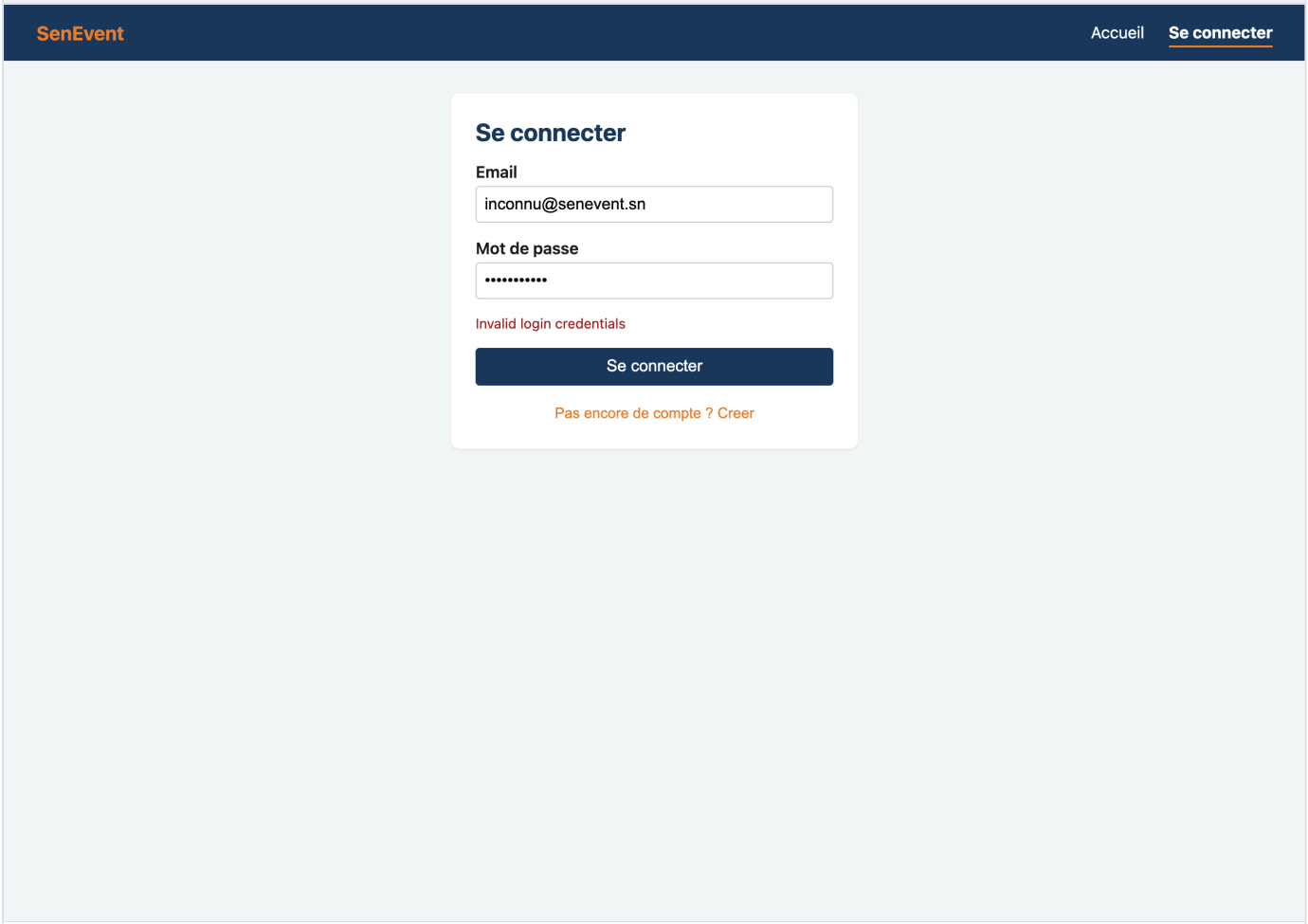
Session active, NavBar contextuelle



Apres le signup, `onAuthStateChange` met a jour l'etat `session` : la NavBar affiche l'email, le bouton « Se deconnecter » et debloque « Nouvel evenement ». Aucun rechargement.

FIGURE 6 `/auth`

Gestion des erreurs



Une tentative de connexion avec des identifiants invalides remonte l'erreur Supabase, affichee en rouge sous le champ. Le `try / catch` du composant capture le message et le rend a l'utilisateur.

Vers le Lab 6

L'authentification passe désormais par Supabase — login, session, déconnexion. Le catalogue, lui, vient encore du mock `evenements.json`. Au Lab 6, `fetch("/evenements.json")` devient `supabase.from('evenements').select()`, le formulaire fait un vrai `INSERT`, l'inscription à un événement écrit dans `inscriptions`, et les images passent par Supabase Storage. La persistance devient réelle : F5 ne perd plus rien.